

 Team 4

MLops

Case

Study

Preetham R	1RV23AI075
Raghavi U Baler	1RV23AI078
Ramita K A	1RV23AI079
Rushil Shodavaram	1RV23AI082

Question 2 :

Using Chapter 5's “Data Access Before Validation and Launch to Production,”

analyze the impact of missing features on model reliability.

IMPACT ↗

- **Prediction instability:** When a trained model receives inputs with missing fields, its internal feature order, scaling, and transformations break. This leads to incorrect or failed predictions.
- **Higher rejection/exception rates:** As seen in MedInsure, 22% of claims are rejected before scoring because the model cannot process incomplete data. This directly affects customer experience.
- **Model bias and inconsistent outputs:** Missing features cause the model to rely on partial information. This results in:
 - Higher false approvals
 - Increased false rejections
 - Skewed risk assessment for specific hospitals that rely on the missing feature
- **Reduced trust and regulatory non-compliance:** Insurance models must demonstrate input completeness and explainability. Missing data leads to unpredictable behavior that auditors will not approve



How MedInsure should implement schema validation before deployment?

Define a formal input schema

- Include:
- Feature names
- Data types
- Allowed value ranges
- Nullable/Non-null constraints
- Categorical domain values

Implement ETL-to-Model Compatibility Checks

- Every ETL update must run automated tests:
- Check all required features exist
- Check feature types and ranges
- Compare current schema with training schema
- Trigger alerts if mismatches occur

Maintain a “Feature Contract”

A feature contract ensures the model and ETL pipeline agree on:

- Feature availability
- Naming conventions
- Preprocessing rules

Question 7 :

Based on all the issues identified, assess whether the model is ready for production. List the mandatory fixes required before deployment.



So is our model ready for production?

Well, absolutely not.

Model in dev: “I got this.”

Model in staging: “What is life?”



How MedInsure should implement mandatory fixes before deployment

Standardize the Environment

- Our dev and staging environments are running on different versions of Python and libraries.
- So The model acts like a different person in each environment.

Fix:

- Same Python version everywhere
- Lock library versions
- Use Docker so the model stops having an identity crisis

Fix Data Pipeline & Implement Schema Validation

- The feature hospital_risk_score vanished in staging.
- The model freaked out and rejected 22% of claims.

Fix:

- Proper schema checks
- Add automated alerts when the ETL pipeline changes.
- Keep ETL versions synced with the ML model

Improve Model Coverage for Rare Surgical Procedures

- The model struggles with rare surgeries because it barely saw them

Fix:

- Add more real or synthetic samples to boost representation.
- Retrain the model with the expanded dataset.
- Recheck performance across all subgroups.
- Log fairness metrics to ensure compliance.



How MedInsure should implement mandatory fixes before deployment

Strengthen System Performance & Handle High Load

- Run load testing with tools
- Monitor latency with APM tools

Fix :

- Horizontal scaling (multiple model-serving replicas)
- Request batching for high traffic
- Async processing / queues for non-urgent predictions
- Model optimization

Ensure End-to-End Reproducibility

- Use MLflow / DVC to track:
- Training data version
- Environment metadata
- Hyperparameters
- Save seeds and training logs to guarantee repeatability.

Improve Security

We need to secure the model so attackers can't manipulate claim data and trick the system into approving high-risk cases as low-risk ones.

Fix:

- Input validation
- Fraud checks
- Alerts for suspicious patterns



“

**Thank
You**

